

Tutorial - Semana 1 - Encontro 1

Introdução

Este é o primeiro tutorial da disciplina Tendências de Motores de Jogos. Antes de tocar em qualquer botão do Unreal Editor, este encontro responde a uma pergunta que vai orientar o semestre inteiro: o que é uma game engine e como ela organiza um mundo jogável? A partir dessa base conceitual, você vai criar o projeto que será a espinha dorsal de todo o semestre — o Vertical Slice *O Templo Esquecido* — e organizar sua estrutura inicial de pastas.

Este tutorial não substitui a explicação do professor em sala. Ele existe para que você possa acompanhar a implementação passo a passo durante o laboratório e revisitar os passos depois da aula, sem depender da documentação oficial da Epic.

Objetivos da Semana

- Compreender o que é uma game engine e por que ela existe, antes de qualquer manipulação de interface.
- Reconhecer as áreas do Unreal Editor (Viewport, Content Browser, Outliner, Details) como instância concreta de conceitos presentes em qualquer engine.
- Criar e organizar a estrutura inicial de pastas do projeto do Vertical Slice, reutilizada em todas as semanas seguintes.

Resultado Esperado ao Final da Semana

Um projeto Unreal Engine 5.6 criado, com estrutura de pastas organizada, e um primeiro Actor Blueprint funcional composto por Components (este segundo resultado é produzido no Encontro 2). Ao final deste Encontro 1 especificamente, você deve ter apenas o projeto criado e a estrutura de pastas organizada — nenhum sistema de gameplay ainda é esperado.

Pré-requisitos

- Conhecimento prévio de desenvolvimento de jogos (você já cursou Programação, Game Design, Unity, IA, Computação Gráfica e Projeto Integrador).
 - Não é necessário conhecimento prévio de Unreal Engine.
-

Antes de começar

O que você deverá possuir antes desta semana

- Unreal Engine 5.6 instalado via Epic Games Launcher.
- Nenhum projeto prévio é necessário — esta semana cria o projeto do zero.

Arquivos necessários

- Nenhum arquivo externo é necessário neste encontro.

Assets utilizados

- Nenhum asset externo é necessário neste encontro. O Kenney Prototype Kit será importado a partir da Semana 3.

Projeto esperado

Ao final deste encontro, o projeto deve se chamar de forma consistente com o Vertical Slice da disciplina (sugestão: `TemploEsquecido`) e conter apenas a estrutura de pastas inicial, sem conteúdo de gameplay.

Parte 1 — O que é uma Game Engine

Objetivo

Compreender a separação entre engine e jogo antes de abrir o editor, para evitar confundir "aprender Unreal" com "aprender a fazer jogos".

Conceito

Uma **game engine** é um software que resolve, de forma reutilizável, um conjunto de problemas comuns a praticamente todo jogo: renderização de gráficos, simulação de física, gerenciamento de

assets, organização de uma cena e execução de lógica de gameplay em tempo real. Sem uma engine, cada estúdio precisaria resolver esses mesmos problemas do zero a cada novo projeto.

O **jogo** é o conteúdo e a lógica específicos construídos sobre a engine: os personagens, as regras, os níveis, a arte. O **editor** é apenas a interface visual que expõe as ferramentas da engine ao desenvolvedor — não é a engine em si, é a porta de entrada para ela.

Essa separação é universal: Unreal, Unity, Godot, O3DE e qualquer outra engine moderna resolvem o mesmo conjunto de problemas, cada uma com decisões arquiteturais próprias. Entender essa distinção é o que vai permitir que você transfira o que aprender aqui para qualquer outro motor no futuro — que é o objetivo real desta disciplina.

Passo a passo

1. Antes de abrir a Unreal, discuta (ou revise mentalmente) a diferença entre engine, editor e jogo.
2. Abrir o Epic Games Launcher.
3. Na aba Unreal Engine, verificar que a versão instalada é a 5.6.
4. Clicar em "Launch" para abrir o Unreal Engine 5.6.
5. Na tela de projetos, selecionar a aba "Games".
6. Escolher o template "Third Person".
7. Selecionar "Blueprint" como tipo de projeto (não C++ — a disciplina permanece em Blueprint Visual Scripting).
8. Definir o local do projeto em uma pasta organizada do seu computador (evite espaços ou acentos no caminho).
9. Nomear o projeto de forma consistente com o Vertical Slice (sugestão: `TemploEsquecido`).
10. Clicar em "Create".

Resultado esperado

O Unreal Editor abre com um projeto Third Person em branco, contendo um personagem controlável em um pequeno cenário de exemplo padrão da Epic.

Verificando

Confirme que o título da janela do editor mostra o nome do projeto escolhido e que o projeto foi criado com o template Third Person em Blueprint (não C++).

Problemas comuns

- **Editor não abre ou trava na criação:** verifique se há espaço em disco suficiente e se a versão 5.6 está corretamente instalada pelo Epic Games Launcher, não uma versão diferente.
- **Template errado selecionado:** se você criou um projeto vazio ("Blank") por engano, feche e recree com o template "Third Person" — ele já traz o Character básico que será reaproveitado nas próximas semanas.
- **Caminho do projeto com acentos ou espaços:** pode causar problemas de compilação e empacotamento mais adiante; recree o projeto em um caminho simples caso identifique isso agora.

Boas práticas

Escolha o local do projeto pensando no semestre inteiro — este mesmo projeto será reaberto e ampliado a cada semana até a Semana 17. Evite pastas temporárias, pendrives ou diretórios sincronizados automaticamente por serviços de nuvem que possam travar arquivos do projeto durante o uso.

Comparação com Unity

Criar um novo projeto na Unreal a partir de um template (Third Person) é conceitualmente equivalente a criar um projeto na Unity a partir de um template do Unity Hub (por exemplo, "3D Core" ou um template de terceira pessoa do Asset Store). Em ambos os casos, o template existe para poupar trabalho inicial de configuração de Player, câmera e input básicos. A diferença é que o template Third Person da Unreal já entrega um Gameplay Framework completo (que exploraremos a partir do Módulo 2), enquanto templates da Unity tendem a ser mais minimalistas.

Parte 2 — Tour pelo Unreal Editor

Objetivo

Reconhecer as áreas principais do Unreal Editor e a função de cada uma, antes de qualquer ação prática.

Conceito

Todo editor de game engine precisa resolver o mesmo problema de interface: mostrar o mundo do jogo, listar os objetos que existem nele, permitir editar as propriedades de cada objeto e dar acesso aos arquivos (assets) do projeto. A forma como cada engine organiza essas quatro necessidades varia, mas o problema resolvido é sempre o mesmo.

Imagem sugerida

Objetivo: mostrar a disposição geral do Unreal Editor com as quatro áreas principais destacadas. Enquadramento: captura de tela cheia do editor logo após a criação do projeto Third Person, com Viewport ao centro. Elementos importantes: Viewport (centro), Content Browser (parte inferior), Outliner (lateral direita, superior), Details (lateral direita, inferior). O que deve ser destacado: contornos coloridos ao redor de cada uma das quatro áreas, com rótulos. Legenda sugerida: "As quatro áreas principais do Unreal Editor: Viewport, Content Browser, Outliner e Details." Referência visual (documentação oficial, apenas para consulta — não copiar a imagem): [Unreal Editor Interface](#)

Passo a passo

1. Localizar o **Viewport**, a área central do editor — é a janela onde o mundo do jogo (o nível/mapa) é visualizado e editado em 3D.
2. Localizar o **Content Browser**, geralmente na parte inferior do editor — é onde ficam todos os arquivos (assets) do projeto: Blueprints, materiais, meshes, mapas.
3. Localizar o **Outliner**, geralmente no canto superior direito — é a lista hierárquica de todos os objetos (Actors) presentes na cena atualmente aberta no Viewport.
4. Localizar o painel **Details**, geralmente abaixo do Outliner — mostra e permite editar as propriedades do objeto atualmente selecionado no Viewport ou no Outliner.
5. Clicar em um objeto qualquer no Viewport (por exemplo, o chão do cenário de exemplo) e observar que ele é destacado simultaneamente no Outliner e que suas propriedades aparecem no painel Details.
6. Navegar pelo Content Browser, abrindo algumas pastas padrão criadas pelo template, sem modificar nada ainda.

Resultado esperado

Você deve ser capaz de identificar rapidamente, sem hesitação, onde encontrar um objeto da cena (Outliner), onde editar suas propriedades (Details), onde visualizar o mundo (Viewport) e onde estão os arquivos do projeto (Content Browser).

Verificando

Peça para um colega apontar aleatoriamente uma das quatro áreas na tela e diga sua função de memória, sem consultar este tutorial.

Problemas comuns

- **Painéis fechados ou fora do lugar:** use o menu Window no topo do editor para reabrir qualquer painel fechado acidentalmente (Outliner, Details, Content Browser).
- **Confundir Outliner com Content Browser:** o Outliner lista o que existe na cena atual; o Content Browser lista todos os arquivos do projeto, estejam eles usados na cena ou não. Essa é uma fonte comum de confusão nas primeiras semanas.

Boas práticas

Redimensione e reorganize os painéis para o layout que for mais confortável para você — a Unreal permite isso livremente, e não há uma disposição "correta" obrigatória, desde que as quatro áreas estejam sempre acessíveis.

Comparação com Unity

O Viewport corresponde à Scene View da Unity; o Content Browser corresponde à Project Window; o Outliner corresponde à Hierarchy; o painel Details corresponde ao Inspector. A lógica de seleção (clicar em um objeto atualiza Hierarchy/Inspector) é idêntica nas duas engines — é uma solução de interface praticamente universal em editores de game engine.

Parte 3 — Estrutura de Pastas do Vertical Slice

Objetivo

Criar a estrutura de pastas inicial do projeto, que será reutilizada e ampliada em todas as semanas seguintes.

Conceito

Organização de Content Browser não é burocracia — é o que permite que um projeto sobreviva a 17 semanas de desenvolvimento incremental sem virar um amontoado de arquivos soltos. Cada engine resolve a organização de assets de forma diferente (pastas físicas na Unreal, pastas físicas também na Unity), mas o princípio de agrupar por função (Blueprints, Materiais, Mapas, dados) em vez de por tipo de arquivo genérico é uma prática comum a qualquer produção profissional.

Passo a passo

1. No Content Browser, clicar com o botão direito na pasta raiz `Content`.
2. Selecionar "New Folder" e criar a subpasta `Blueprints`.
3. Dentro de `Blueprints`, criar as subpastas `Characters`, `Interactables`, `Framework` e `Components` (estas duas últimas serão usadas a partir da Semana 4, mas já organizamos a estrutura agora).
4. Na raiz de `Content`, criar a pasta `Maps` e, dentro dela, as subpastas `Exploration` e `Dungeon`.
5. Na raiz de `Content`, criar a pasta `Environment` e, dentro dela, as subpastas `Dungeon` e `Nature`.
6. Criar também na raiz: `Characters` (para Skeletal Mesh e Animation Blueprints, usada a partir do Módulo 3), `UI`, `Materials` (com subpastas `Base` e `Instances`), `Audio`, `Data` (com subpastas `DataTables`, `DataAssets` e `Structs_Enums`), `Textures` e `Meshes`.
7. Mover o mapa de exemplo gerado pelo template Third Person para dentro de `Maps/Exploration`, renomeando-o para `Map_Exploration`.

Resultado esperado

O Content Browser deve exibir a estrutura de pastas completa prevista para o Vertical Slice, mesmo que a maioria delas esteja vazia por enquanto — elas serão preenchidas ao longo do semestre.

Verificando

Compare sua estrutura de pastas com a referência abaixo. Nenhum asset deve estar solto diretamente na raiz de `Content`.

Pasta	Uso
<code>Blueprints/Characters</code>	BP_Player, BP_Enemy (a partir da Semana 2 e do Módulo 3)

Blueprints/Interactables	BP_Door, BP_Lever, BP_Chest, BP_Pickup, BP_Checkpoint (a partir do Módulo 2)
Blueprints/Framework	BP_GameMode, BP_GameState, BP_PlayerController, BP_GameInstance (a partir do Módulo 2)
Blueprints/Components	InteractionComponent, InventoryComponent, HealthComponent, SaveComponent
Maps/Exploration , Maps/Dungeon	Mapas do Vertical Slice
Environment/Dungeon , Environment/Nature	Assets do Kenney (a partir da Semana 3)
Data/DataTables , Data/DataAssets , Data/Structs_Enums	Dados de design desacoplados (a partir do Módulo 2)

Problemas comuns

- **Renomear arquivo sem usar o editor:** nunca renomeie ou mova arquivos da Unreal pelo explorador de arquivos do sistema operacional — isso quebra referências internas. Sempre use o Content Browser.
- **Pastas soltas ou duplicadas:** se perceber que criou uma pasta com nome duplicado ou em local errado, mova-a (arrastando dentro do Content Browser) em vez de recriar do zero.

Boas práticas

Nomeie pastas e mapas seguindo as convenções que serão usadas o semestre inteiro (por exemplo, prefixo `Map_` para mapas). Adotar a convenção desde a primeira semana evita retrabalho de renomeação mais tarde, quando o projeto já tiver dezenas de referências cruzadas entre Blueprints.

Comparação com Unity

A ideia de organizar `Assets/` por função (Scripts, Prefabs, Materials, Scenes) na Unity é o mesmo princípio aplicado aqui à pasta `Content/` da Unreal. A diferença prática é que a Unreal já sugere fortemente esse padrão de nomenclatura com prefixos (`BP_` , `M_` , `WBP_`), enquanto na Unity a convenção de nomes é definida inteiramente pela equipe, sem um padrão imposto pela engine.

Ao final da semana

Este Encontro 1 cobre a primeira metade da Semana 1. Ao final dele, o projeto deve estar criado com o template Third Person, em Blueprint, com a estrutura de pastas completa organizada conforme a tabela acima. Nenhum Actor customizado, sistema de gameplay ou build ainda são esperados — isso é produzido no Encontro 2 e nas semanas seguintes, conforme o roadmap do PROJECT_ARCHITECTURE.md (Módulo 1 — Semanas 1 a 3).

Desafio

Explore o cenário de exemplo do template Third Person no modo Play (tecla Play no topo do editor) e identifique, sem ajuda, pelo menos três elementos do mundo que você consegue localizar no Outliner. Anote os nomes desses três Actors — eles serão usados como referência de comparação quando criarmos nosso primeiro Actor customizado no próximo encontro.

Checklist

- ☐ Unreal Engine 5.6 aberta com o projeto `TemploEsquecido` (ou nome equivalente) criado a partir do template Third Person em Blueprint
- ☐ Consigo identificar de memória as quatro áreas do editor: Viewport, Content Browser, Outliner, Details
- ☐ Estrutura de pastas completa criada dentro de `Content/`, conforme a tabela de referência
- ☐ Mapa de exemplo movido e renomeado para `Map_Exploration` dentro de `Maps/Exploration`
- ☐ Nenhum asset solto na raiz de `Content/`

Glossário

- **Engine:** software que fornece as ferramentas e o runtime para construir e executar um jogo.
- **Editor:** interface visual que expõe as ferramentas da engine ao desenvolvedor.
- **Viewport:** área do editor onde o mundo do jogo é visualizado e editado em 3D.
- **Content Browser:** área do editor que lista todos os arquivos (assets) do projeto.
- **Outliner:** lista hierárquica dos objetos (Actors) presentes na cena atual.
- **Details:** painel que exibe e permite editar as propriedades do objeto selecionado.
- **Actor:** unidade básica de qualquer objeto que pode existir em um nível da Unreal (será aprofundado no Encontro 2).

Referências

- EPIC GAMES. **Unreal Engine 5 Documentation** — Visão geral do Editor. Disponível em: <https://dev.epicgames.com/documentation/en-us/unreal-engine>.
- EPIC GAMES. **Unreal Engine Learning Library**. Disponível em: <https://dev.epicgames.com/community/unreal-engine/learning>.
- UNITY TECHNOLOGIES. **Unity Manual** — Interface do Editor, para fins comparativos. Disponível em: <https://docs.unity3d.com/Manual/>.
- Vídeo sugerido (apoio complementar, nunca substituindo a documentação oficial): canal oficial **Unreal Engine**, vídeos introdutórios de tour pelo editor.